# Lecture 13 Crash Study Guide

## File Management - Operating Systems CS x61

*Emergency exam guide: what to understand, what to memorize, and how to answer questions*

**Study order for the next few hours**

- First master file organization types: pile, sequential, indexed sequential, indexed, hashed.
- Then master indexes: primary vs clustering vs secondary, dense vs sparse, single-level vs multi-level.
- Then master storage: blocking, contiguous/chained/indexed allocation, free-space management.
- Finally memorize UNIX inode structure and NTFS features/components.

## 0. Lecture map and exam priorities

| Slide range | Topic | Exam value |
|---|---|---|
| 2-6 | What files are, FMS goals, minimum requirements, fields/records/files/databases | Definitions and short-answer questions |
| 7-13 | File system architecture and elements of file management | Layering diagram and responsibility questions |
| 14-42 | File organization: pile, sequential, indexed sequential, indexed, direct/hashed, indexes | Highest value: comparisons, pros/cons, numerical index questions |
| 43-52 | Directories, naming, sharing, access rights, simultaneous access | Concept and permission questions |
| 53-56 | Records vs blocks and blocking methods | Traps about logical vs physical units and fragmentation |
| 57-68 | Secondary storage management and file allocation | Contiguous/chained/indexed allocation comparisons |
| 69-73 | Free-space management | Bit table, free list, indexing free space |
| 74-81 | UNIX/FreeBSD file management and inodes | Inode, direct/indirect pointers, directory lookup |
| 82-86 | Windows NTFS | NTFS features, MFT, components, recovery |

**One-sentence lecture summary**

- A file system is the OS layer that turns raw secondary storage blocks into named, protected, sharable files, while managing directories, file organization, allocation, free space, and reliability.

## 1. Files and File Management System

- **File:** a named collection of related information stored for long-term use. Unlike process memory, it remains after logout or program termination.
- **File system:** the OS abstraction over secondary storage. Users see files and directories, not raw disk sectors.
- **FMS:** file management system. It consists of privileged OS utilities that create, delete, open, close, read, and write files.
- **Why important:** almost every application receives input from files and saves output to files.

| File operation | Meaning |
|---|---|
| Create | Define a new file and place it inside the file structure. |
| Delete | Remove the file from the structure and destroy it. |
| Open | Allow a process to perform operations on the file. |
| Close | End the process relation to the file until it is opened again. |
| Read | Copy all or part of file data into the process. |
| Write | Update the file by changing existing data or adding new data. |

## 2. Objectives and minimum requirements

- Meet user data-management needs.
- Guarantee file data validity.
- Optimize performance: throughput for the system and response time for users.
- Support many storage-device types through standardized I/O routines.
- Minimize lost or destroyed data.
- Support multiple users when needed.

### Minimum requirements to memorize

- User can create, delete, read, write, and modify files.
- User can control access to their files and possibly access other users files with permission.
- User can restructure files, move data between files, backup and recover files.
- User accesses files by symbolic names, not numeric disk identifiers.

## 3. Core data terms

| Term | Definition | Example |
|---|---|---|
| Field | Basic data element containing one value. Has length and data type. | Name, date, salary |
| Record | Collection of related fields treated as one unit. Fixed or variable size. | Employee record |
| File | Collection of similar records. Referenced by name; access control often applies here. | Employees file |
| Database | Collection of related data with relationships among elements. Often managed by a DBMS outside the OS. | University DB |

## 4. File system architecture layers

Think of the architecture as a stack. The upper layers understand records and user operations; the lower layers understand blocks and devices.

| Layer | Job | Professor trap |
|---|---|---|
| Access method | Closest to the user. Provides standard interface and depends on file structure: sequential, indexed, hashed, etc. | If asked about user-visible access style, answer access method. |
| Logical I/O | Deals with records, not raw blocks. Maintains basic file data and record I/O. | Logical I/O = records. |
| Basic I/O supervisor | Initiates/terminates file I/O, maintains control structures, chooses device, schedules disk/tape I/O, assigns buffers and secondary storage. | This is where file I/O initiation and termination happen. |
| Basic file system / physical I/O | Deals with blocks on disk/tape, block placement, and buffering in main memory. Does not understand file contents. | Blocks, not records. |
| Device drivers | Lowest layer. Communicates directly with controllers/devices and starts/completes device I/O. | Drivers are usually part of the OS. |

### Architecture memory hook

- Top to bottom: Access method -> Logical I/O -> Basic I/O supervisor -> Basic file system -> Device drivers.
- Records are handled above; blocks and devices are handled below.

## 5. File organization: the biggest exam section

File organization means the logical structure of records and how records are accessed. Do not confuse it with physical disk allocation, which comes later.

| Criterion | Meaning |
|---|---|
| Rapid access | How fast records can be found. |
| Ease of update | How easy insert/delete/modify operations are. |
| Economy of storage | How much storage overhead or redundancy is needed. |
| Simple maintenance | How easy the structure is to maintain over time. |
| Reliability | How robust the structure is against errors. |

## 6. Five file organization types

| Type | How it works | Strengths | Weaknesses / traps |
|---|---|---|---|
| Pile / heap | Records are stored in arrival order. New records go at the end. Records may be variable length and may have different fields. | Very easy to update. Good space use for variable records. | Search is exhaustive linear search. Average search reads about half the file. Sorting needed to read by a field. |

| | | | |
|---|---|---|---|
| Sequential | Fixed-format records, same length, same fields. Records are stored in key/order sequence. | Efficient ordered reading. Binary search possible on ordering key. | Insertion is expensive. Random requests are poor unless using the ordering key. Often uses overflow/transaction file for new records. |
| Indexed sequential | Sequential file plus an index plus overflow area. | Keeps sequential processing and supports faster random lookup by key. | Best processing is based on one key field. Overflow must be periodically merged. |
| Indexed | Sequentiality abandoned. Records accessed through one or more indexes. Variable-length records possible. | Fast access using many fields. Multiple indexes allow many access paths. | Adding a record requires updating all indexes. Extra storage overhead. |
| Direct / hashed | Hash function maps key value K to bucket h(K). | Very rapid single-record access by hash key. | Bad for ordered access. Collisions and overflow must be handled. Static bucket count is bad if file grows/shrinks. |

### Fast selection logic for exam MCQs

- Need easiest insert and no ordering? Pile.
- Need ordered processing by one key? Sequential.
- Need ordered file plus random lookup on key? Indexed sequential.
- Need many search fields? Indexed file.
- Need fastest single-record lookup by key? Hashed/direct file.

## 7. Indexes and access paths

- **Index:** an auxiliary file that makes searching for records faster.
- **Access path:** an index on a field. It maps a field value to a pointer to a record or block.
- **Dense / exhaustive index:** has an index entry for every search-key value, usually every record.
- **Sparse / nondense / partial index:** has entries only for some search values, often one per block or one per distinct field value.

| Index type | Defined on | Entries | Dense or sparse? | Key idea |
|---|---|---|---|---|
| Primary | Ordered data file on a key field | One entry per data block, usually the first key value in the block (block anchor) | Sparse / nondense | Fast search on ordering key; block anchoring yes. |
| Clustering | Ordered data file on a non-key field | One entry per distinct value of the field | Sparse / nondense | Records with the same non-key value are grouped together. |
| Secondary key | Non-ordering candidate key field | One entry per record | Dense | Alternative access path; no block anchoring. |
| Secondary non-key | Non-ordering field with duplicate values | Entries may be per record or per distinct value with lists | Dense or sparse | Useful for searching fields other than the primary ordering field. |

## 8. Index numerical questions

These formulas are important because the lecture gives an index calculation example. Memorize the steps.

**Blocking factor:** `Bfr = floor(B / R)`

*B = block size, R = record size. It tells how many records fit in one block.*

**Number of data blocks:** `b = ceil(r / Bfr)`

*r = number of records.*

**Index entry size:** `RI = V + P`

*V = index field size, P = pointer size.*

**Index blocking factor:** `BfrI = floor(B / RI)`

*How many index entries fit in one index block.*

**Number of index blocks:** `bI = ceil(r / BfrI)`

*For dense indexes with one entry per record. Sparse indexes may use fewer entries.*

**Binary search cost:** `ceil(log2(number_of_blocks)) block accesses`

*Use data blocks for ordered file search; use index blocks for index search.*

| Lecture example value | Calculation | Result |
|---|---|---|
| B=512, R=150, r=30000 | Bfr=floor(512/150) | 3 records/block |
| Data file blocks | b=ceil(30000/3) | 10000 blocks |
| VSSN=9, pointer=7 | RI=9+7 | 16 bytes/index entry |
| Index entries per block | BfrI=floor(512/16) | 32 entries/block |
| Index blocks | bI=ceil(30000/32) | 938 blocks |
| Binary search on index | ceil(log2 938) | 10 block accesses |
| Average linear search | 10000/2 | 5000 block accesses |
| Binary search on ordered data file | ceil(log2 10000) | 14 block accesses |

**Trap**

- Use floor for records per block because a partial record cannot count as a full record.
- Use ceil for number of blocks because leftover records still require a block.
- An index can be much smaller than the data file because index entries are smaller than full records.

## 9. Multi-level indexes

- A single-level index is an ordered file. Because it is ordered, we can build another index on top of it.
- The original index is the first-level index; the index on it is the second-level index.
- Repeat until the top-level index fits in one disk block.
- This works for primary, clustering, or secondary indexes as long as the first-level index has more than one disk block.

## 10. Direct / hashed files

- A direct file accesses a block at a known address.
- A hashed file uses a hash function h(K) to map a key K to bucket i.
- For disk files, hashing is called external hashing.
- File blocks are divided into M equal-sized buckets. A bucket is usually one or a fixed number of disk blocks.
- Search by hash key is very efficient, but ordered access is poor because records are not stored in key order.

| Collision method | Meaning |
|---|---|
| Open addressing | If the computed position is occupied, check subsequent positions until an empty position is found. |
| Chaining | Use overflow locations and pointer fields. New colliding records go to overflow and are linked from the bucket. |
| Multiple hashing | Apply another hash function after collision; if still collision, continue or use open addressing. |

### Hashed-file traps

- Hash files are usually kept about 70-80 percent full to reduce overflow.
- The hash function should distribute records uniformly; otherwise overflow chains become long.
- Static hashing has a fixed number of buckets, so growth or shrinkage is a problem.
- Hashed files are not good for ordered access on the hash key; sorting would be needed.

## 11. Directories

- A directory contains information about files: names, attributes, ownership, location, and access permissions.
- The directory itself is a file owned by the operating system.
- A directory maps human-readable file names to the actual files or to metadata structures such as inodes.

| Directory element | Examples |
|---|---|
| Basic information | File name, file type, file organization. |
| Address information | Volume/device, starting address, size used, size allocated. |
| Access control information | Owner, authorized users, passwords or access rights, permitted actions. |
| Usage information | Creation time, last read, last modified, last backup, locks/current activity. |

| Directory operation | Meaning |
|---|---|
| Search | Find the entry matching a referenced file. |
| Create file | Add a new directory entry. |
| Delete file | Remove a directory entry. |
| List directory | Show files owned by a user and some attributes. |

| | |
|---|---|
| Update directory | Change directory entry when file attributes change. |

## 12. Directory structures and naming

| Structure | Description | Weakness / exam note |
|---|---|---|
| Simple directory | One list of entries, one per file. File name can be the key. | No help organizing files. File names must be globally unique. |
| Two-level directory | Master directory has one entry per user. Each user has their own file directory. | Avoids conflicts between users but does not structure a users own files well. |
| Tree / hierarchical directory | Master directory -> user directories -> subdirectories -> files. | Most flexible. Names unique only within the directory. Large directories may use hashed files for performance. |

- **Path:** sequence of directories from the root/master directory to the file, e.g. /UserB/Word/UnitA/ABC.
- **Working directory:** current directory used to interpret relative paths.
- **File names:** not necessarily globally unique. Pathnames are unique because they include the directory path.

## 13. File sharing and access rights

- Multiuser systems should allow controlled file sharing.
- Two issues: access rights and simultaneous access.

| Right | Meaning |
|---|---|
| None | Others may not even know the file exists. |
| Knowledge | User knows the file exists and who owns it. |
| Execution | User can execute the file if it is a program. |
| Read | User can view or copy contents. |
| Append | User can add data but not modify existing data. |
| Update | User can modify, delete, and add data. |
| Change protection | User can grant or change access rights. |
| Deletion | User can delete the file. |

### Simultaneous access

- When many users access or modify the same file, the OS/FMS must enforce discipline.
- This is an instance of the readers-writers problem.
- The system must consider mutual exclusion and deadlock.
- Locking the entire file is simpler; locking individual records gives more concurrency.

## 14. Blocks, records, and blocking

- **Record:** logical unit of access in a structured file.
- **Block:** physical unit of I/O with secondary storage.
- Records must be organized into blocks before disk I/O can happen.
- Large blocks transfer more records per I/O and can reduce access time, but waste space and time for random reads and require larger buffers.

| Blocking method | How it works | Good | Bad |
|---|---|---|---|
| Fixed blocking | Fixed-length records. Integral number of records per block. | Easy to find arbitrary records. | Unused space at block end = internal fragmentation. |
| Variable spanned | Variable-length records packed tightly; a record may span multiple blocks. | No unused block space; record can be bigger than a block. | May need to read multiple blocks; updates are difficult. |
| Variable unspanned | Variable-length records, but a record cannot span blocks. | Simpler than spanned. | Wastes space in most blocks; record size limited to block size. |

## 15. Secondary storage management and file allocation

- The OS allocates disk blocks to files and tracks free disk space.
- Questions: allocate maximum space when file is created? What portion size? What data structure tracks portions? Example: FAT.

| Policy | Meaning | Pros / cons |
|---|---|---|
| Preallocation | Allocate maximum file size at creation, usually contiguous. | Fast access and good contiguity, but requires knowing max size and often wastes space. |
| Dynamic allocation | Allocate portions as file grows. | Avoids overestimating size, but file may become non-contiguous. |

### Portion-size tradeoff

- Large contiguous portions improve performance and reduce allocation-table size, but can waste space and cause fragmentation.
- Small blocks are flexible and reduce wasted space, but require larger allocation tables and less locality.
- Fixed-size portions simplify reallocation; variable-size portions reduce wasted space but require handling fragmentation.

## 16. Three file allocation methods

| Method | How it works | Advantages | Disadvantages |
|---|---|---|---|
| Contiguous | One contiguous set of blocks allocated at creation. File table stores starting block and length. | Excellent sequential performance; multiple blocks can be read at once; simple table entry. | Needs predeclared size; external fragmentation; compaction may be needed. |
| Chained | File stored as linked blocks. Each block points to the next. File table stores starting block and length. | No external fragmentation; no preallocation; any free block can be added. | Direct access is slow because you trace the chain; no locality; pointer overhead; FAT variant stores links in |

| | | | |
|---|---|---|---|
| | | | memory table. |
| Indexed | File table points to an index block; index block points to all data blocks. | Supports sequential and direct access; common; Unix uses a variation. | Index overhead; may still need consolidation; fixed blocks eliminate external fragmentation but variable portions improve locality. |

### Allocation memory hook

- Contiguous = fastest but fragments and needs size.
- Chained = flexible but bad direct access/locality.
- Indexed = balanced; supports direct and sequential access.

## 17. Free-space management

| Method | How it works | Strengths | Weaknesses |
|---|---|---|---|
| Bit table / bitmap | One bit per disk block: 0 free, 1 used. | Smallest representation; easy to find free blocks or contiguous group; works with any allocation method. | Can still be large; exhaustive search can slow performance. |
| Chained free portions | Free portions linked using pointer and length stored in each free portion. | Negligible space overhead; suitable for all allocation methods. | Can fragment badly; slow creation/deletion for many small blocks. |
| Indexing free space | Treat free space like a file with an index table, preferably variable-size portions. | Efficient support for all file allocation methods. | Index table overhead. |
| Free block list | List numbers of all free blocks in reserved disk area. Can use FIFO or stack behavior. | Can reallocate recently freed blocks with only a few kept in memory. | Much larger than bitmap: 24 or 32 bits per block number, so usually stored on disk. |

**Bitmap size formula:** `bitmap bytes = number_of_blocks / 8 = (disk_size / block_size) / 8`

### Lecture example: 16 GB disk, 512-byte blocks

- Number of blocks = 16 GB / 512 bytes = 33,554,432 blocks if using binary GB.
- Bitmap needs one bit per block = 33,554,432 bits = 4,194,304 bytes = about 4 MB.
- This is why a bit table is small relative to disk size, but it can still be significant in main memory.

## 18. UNIX and FreeBSD file management

| UNIX file type | Meaning |
|---|---|
| Regular / ordinary | User, application, or system-utility information. |
| Directory | List of file names with pointers to inodes. |
| Special | Used to access peripheral devices such as terminals or printers. |
| Named pipe | Interprocess I/O. |

| | |
|---|---|
| Link | Alternative file name for an existing file. |
| Symbolic link | A data file containing the name/path of another file. |

- **Inode:** index node. A control structure holding key information for one file.
- Several file names may be associated with one inode, but each file is controlled by one inode.
- The inode stores attributes, permissions, ownership, timestamps, file size, and block pointers.
- Directories are files containing file names and pointers to inodes.

| Inode field / idea | What to remember |
|---|---|
| Mode/type/access mode | What kind of file it is and how it can be accessed. |
| Owner/group IDs | Used for permissions. |
| Timestamps | Creation and last read/write times. |
| File size | Size of file. |
| Block pointers | Pointers to data blocks and indirect blocks. |
| Reference count / directory entries | Important for links and deletion behavior. |
| Flags / generation / extended attributes | Control and metadata fields. |

**UNIX inode allocation trap**

- File allocation is dynamic and block-based; blocks do not have to be contiguous.
- Part of the index is stored in the inode itself.
- Inode has direct pointers for small files, then single indirect, double indirect, and triple indirect pointers for larger files.
- Direct pointer points directly to data; single indirect points to a block of pointers; double indirect points to blocks of pointers to pointer blocks; triple indirect adds one more level.

## 19. Looking up a UNIX pathname

For a path like /usr/ast/mbox, the OS starts at the root directory, finds usr, uses its inode to open the usr directory, finds ast, uses its inode, then finds mbox and its inode. Every directory is just a file mapping names to inode numbers.

## 20. Windows NTFS

| NTFS feature | Meaning |
|---|---|
| Large disks and large files | Designed for large storage. |
| Multiple data streams | A file can have more than one stream of data. |
| Journaling | A log records changes to support recovery. |
| Compression/encryption | Built-in space saving and security features. |
| Recoverability | Transaction-like model: changes treated as atomic actions. |
| Security | Access-control support. |

| NTFS storage unit | Meaning |
|---|---|
| Sector | Smallest physical storage unit, usually 512 bytes. |
| Cluster | One or more contiguous sectors. Fundamental NTFS allocation unit. Can be 1 to 128 sectors. |
| Volume | Logical partition on disk; may be part of a disk, whole disk, or span disks with RAID. |

| NTFS volume layout item | Meaning |
|---|---|
| Partition boot sector | First sectors; volume layout, file system structure, boot startup. |
| Master File Table (MFT) | Information about all files and directories. Table rows are 1024 bytes; each row describes a file and attributes. |
| System files | Log file, cluster bitmap, attribute definition table. |
| Files | User and system file contents. |

**NTFS component trap**

- I/O manager handles basic open, close, read, write, and software RAID.
- Log file service keeps log of disk writes for crash recovery.
- Cache manager optimizes disk I/O using lazy write and lazy commit.
- Virtual memory manager maps file references to virtual memory and accesses cached files.
- NTFS recovery mainly protects system metadata, not necessarily user data.

## 21. High-yield comparison tables

| Question | Best answer |
|---|---|
| Fastest single-record lookup by exact key? | Hashed / direct file. |
| Best for sequential ordered processing? | Sequential or indexed sequential. |
| Best when searching by many fields? | Indexed file with multiple indexes. |
| Easiest update/insertion? | Pile, but search is poor. |
| Best allocation performance for sequential access? | Contiguous allocation. |
| Allocation with no external fragmentation? | Chained allocation or indexed allocation with fixed-size blocks. |
| Allocation that supports direct and sequential access? | Indexed allocation. |
| Free-space method with one bit per block? | Bit table / bitmap. |
| Unix metadata structure? | Inode. |
| NTFS central metadata table? | Master File Table (MFT). |

## 22. Professor traps and exact lines to remember

**Traps**

- File organization is logical record structure; file allocation is physical storage placement.
- Basic file system handles blocks, not records. Logical I/O handles records.
- A directory is itself a file owned by the OS.
- A path is unique; a file name alone does not have to be globally unique.
- Primary index is sparse/nondense and uses block anchors.
- Secondary index is dense when it has one entry per record.
- Sequential file has efficient ordered access but expensive insertion.
- Hashed file gives fast exact-key lookup but poor ordered access.
- Chained allocation eliminates external fragmentation but is bad for direct access.
- Record locking gives more concurrency than whole-file locking, but is more complex.

## 23. Likely exam questions with model answers

**Q: Define a file and list desirable file properties.**

A: A file is a named collection of related information stored on secondary storage. Desirable properties include long-term existence, sharing between processes/users through names and permissions, and structure both internally and through directories.

**Q: What is the difference between logical I/O and the basic file system?**

A: Logical I/O deals with file records and record-level access. The basic file system deals with blocks on storage devices, block placement, and buffering, without understanding file contents.

**Q: Compare pile and sequential files.**

A: Pile stores records as they arrive and is easy to update but needs linear search. Sequential stores fixed-format records ordered by key; ordered reading and binary search by key are efficient, but insertion is expensive.

**Q: Why does indexed sequential improve sequential files?**

A: It keeps records ordered by key but adds an index for faster random access and an overflow area for new records.

**Q: What is the difference between primary, clustering, and secondary indexes?**

A: Primary index is on an ordered key field and is sparse with block anchors. Clustering index is on an ordered non-key field and has one entry per distinct value. Secondary index is an alternative access path on a non-ordering field and is often dense.

**Q: Why can a multi-level index be created?**

A: Because an index is itself an ordered file, so we can index the index until the top level fits in one block.

**Q: What is external hashing?**

A: Hashing for disk files. Records are stored in buckets based on h(K), where K is the hash key. Collisions use overflow, chaining, open addressing, or multiple hashing.

**Q: What does a directory do?**

A: It maps file names to file metadata/location and stores information such as attributes, ownership, access rights, and usage information.

**Q: Why is simultaneous file access a synchronization issue?**

A: Multiple users may read/write the same file. It is a readers-writers problem requiring mutual exclusion, deadlock awareness, and locking at file or record level.

**Q: Compare contiguous, chained, and indexed allocation.**

A: Contiguous is fast and simple but needs preallocation and causes external fragmentation. Chained is flexible and avoids external fragmentation but direct access is slow. Indexed uses an index block to support direct and sequential access, with index overhead.

**Q: What is an inode?**

A: An inode is a UNIX/FreeBSD control structure containing file metadata such as type, permissions, owner, timestamps, size, and block pointers.

**Q: What is the role of NTFS journaling?**

A: It records changes in a log so that NTFS can recover file-system metadata after crashes.

## 24. Practice numerical question

Question: A file has 50,000 records. Record size is 200 bytes. Block size is 1,000 bytes. An index is built on a field of 10 bytes with a 6-byte pointer. Estimate data blocks, index blocks, and binary search cost on the index.

| Step | Calculation | Answer |
|---|---|---|
| Data blocking factor | floor(1000 / 200) | 5 records/block |
| Data blocks | ceil(50000 / 5) | 10000 blocks |
| Index entry size | 10 + 6 | 16 bytes |
| Index blocking factor | floor(1000 / 16) | 62 entries/block |
| Index blocks | ceil(50000 / 62) | 807 blocks |
| Binary search on index | ceil(log2 807) | 10 block accesses |

## 25. Flashcards

| Term | Answer | Term | Answer |
|---|---|---|---|
| FMS | Privileged OS utilities/services for create/delete/open/close/read/write files. | Field vs record | Field = one value. Record = related fields as one unit. |
| Logical I/O | Deals with records. | Basic file system | Deals with blocks, placement, buffering. |
| Pile weakness | Linear/exhaustive search. | Sequential weakness | Expensive insertion and poor random requests. |

| | | | |
|---|---|---|---|
| Indexed sequential additions | Index + overflow area. | Dense index | One entry per search key value / usually per record. |
| Sparse index | Entries for only some search values / often one per block. | Primary index | Ordered key field, sparse, block anchor. |
| Clustering index | Ordered non-key field, one entry per distinct value. | Secondary index | Alternative access path; often dense. |
| External hashing | Disk-file hashing into buckets. | Directory | OS-owned file mapping file names to files/metadata. |
| Path | Sequence of directories to reach a file. | Access rights | None, knowledge, execution, read, append, update, change protection, deletion. |
| Fixed blocking | Fixed records, integral records per block, internal fragmentation. | Spanned blocking | Variable records can cross blocks. |
| Unspanned blocking | Variable records cannot cross blocks. | Contiguous allocation | Start block + length, fast but external fragmentation. |
| Chained allocation | Linked blocks, no external fragmentation, bad direct access. | Indexed allocation | Index block points to data blocks. |
| Bitmap | One bit per block: 0 free, 1 used. | Inode | UNIX file metadata/control structure. |
| MFT | NTFS Master File Table: metadata for files and directories. | | |

## 26. Last 30-minute checklist

- Recite architecture stack from top to bottom and say what each layer handles.
- Compare the five file organizations without looking.
- Memorize dense vs sparse and primary vs clustering vs secondary indexes.
- Do the index calculation formulas once: Bfr, blocks, index entry size, index blocks, log2 search cost.
- Compare contiguous/chained/indexed allocation in one sentence each.
- Compare bit table/chained free portions/indexing/free block list.
- Say how UNIX inode direct/single/double/triple indirect pointers work.
- Memorize NTFS: MFT, journaling/log, cache manager, VMM, recovery of metadata.